

PEZD: A practical and effective zero-delay defense against website fingerprinting

Hengheng Xiong^{ID}, Dapeng Man^{ID*}, Huanran Wang^{ID}, Jingwen Tan^{ID}, Jiguang Lv, Wu Yang

College of Computer Science and Technology, Harbin Engineering University, Harbin, 150001, Heilongjiang, China

ARTICLE INFO

Keywords:

Privacy
Traffic analysis
Anonymity system
Website fingerprinting defense

ABSTRACT

Website Fingerprinting (WF) enables an attacker to infer which website a user is visiting by analyzing the side-channel information of network traffic, posing a serious threat to anonymous systems such as Tor. To mitigate this issue, numerous defenses have been proposed to resist WF attacks. However, many defenses incur high overhead, impeding their deployment. Moreover, some defenses rely on the impractical assumption that the defender knows prior knowledge of which website the user visits. In this paper, we propose a practical and effective lightweight WF defense, named PEZD. It hinders attackers from recognizing a website's unique patterns by adding dummy packets with diverse distributions into traffic traces. In particular, PEZD is a website-agnostic defense that does not require knowledge of the website users are visiting and incurs only moderate overhead to effectively prevent attacks. Extensive experiments demonstrate that PEZD reduces the accuracy of state-of-the-art attacks from 97 % to 19%–38 % while introducing zero latency overhead and only 48 % bandwidth overhead. Moreover, we show that PEZD remains effective for injecting dummy packets only on the client-side, simplifying the implementation of the defense.

1. Introduction

An increasing number of people are using privacy-enhancing tools such as Tor [1] to access the Internet, aiming to seek protection against the potential risks associated with network surveillance. Tor is the most famous anonymous networking service, with two million people using it every day [2,3]. It relays user traffic through multiple volunteer nodes, such that attackers cannot capture both their identity and online activity simultaneously. Nevertheless, many studies have shown that the privacy of Tor has been undermined by Website Fingerprinting (WF) attacks [4–13]. In particular, WF attacks are traffic analysis attacks that allow an attacker to identify which websites a user visits by analyzing the side-channel information of network traffic, i.e., directions, sizes, and timing of packets. Such information can be utilized to build unique patterns for a target website. The attacker identifies which websites a user visits by recognizing unique patterns in network traffic.

To mitigate WF attacks, numerous defenses have been proposed [4,6,14–28]. Specifically, WF defenses reshape network traffic by injecting dummy network packets and/or delaying network packets. Among them, BuFLO [19], Tamaraw [20], and CS-BuFLO [21] mimic a constant transmission rate, but they incur bandwidth and latency overheads several times higher than the original traffic, severely hindering their deployment. To mitigate the high overhead, researchers have proposed a

variety of lightweight (moderate overhead) defenses. However, many defenses [22–24,28,29] rely on the impractical assumption that the defender knows prior knowledge of which website the user visits, so that they can collect traffic traces from these websites to generate adversarial perturbations. The disadvantage of this type of defense is that it only protects specific websites and lacks defense generality. Some website-agnostic lightweight defenses have been proposed [17,27,30–32], but several of these defenses [30,31] require to delay packet, resulting in potential out-of-memory errors on Tor's relays. Moreover, some defenses have been undermined by state-of-the-art WF attacks [9,10], such as WTF-PAD [17] and FRONT [27]. In particular, FRONT injects dummy packets according to a Rayleigh distribution to obfuscate the front of network traffic since WF attacks rely on feature-rich trace fronts. We show that attackers can still identify the defended traffic traces by exploiting the features present in the latter part of the traces, with the Var-CNN attack [10] achieving an accuracy of 80%. Furthermore, even when TBP [32] employs multiple fixed distributions for dummy packet injection, Var-CNN can still achieve 70% accuracy against it. These results demonstrate the fundamental vulnerability of defenses using deterministic distributions.

To address the aforementioned limitations, we propose PEZD, a practical and effective lightweight WF defense. The basic idea is to hinder an attacker from discovering patterns common to intra-website traffic

* Corresponding author.

E-mail addresses: xionghengheng@hrbeu.edu.cn (H. Xiong), mandapeng@hrbeu.edu.cn (D. Man), huanran.wang@hrbeu.edu.cn (H. Wang), ttt19@hrbeu.edu.cn (J. Tan), lvjiguang@hrbeu.edu.cn (J. Lv), yangwu@hrbeu.edu.cn (W. Yang).

<https://doi.org/10.1016/j.comnet.2025.111716>

Received 23 May 2024; Received in revised form 21 April 2025; Accepted 11 September 2025

Available online 14 September 2025

1389-1286/© 2025 Published by Elsevier B.V.

traces by injecting dummy packets with diverse distributions for each load. To the best of our knowledge, this paper shows for the first time that WF attacks can be undermined by injecting dummy packets with diverse distributions. Specifically, PEZD utilizes the unique nature of the distribution of outgoing packets from different websites to construct the distribution of dummy packets so that the distribution of dummy packets sent by the same website varies for each load. The diversity in the distribution of dummy packets diversifies the traffic traces of users on each visit, resulting in the existence of fewer common traffic patterns on the same websites, which severely interferes with the learning process of the attack model. The distribution of dummy packets can be generated from the traffic trace of any website and therefore can be pre-generated and applied to real-time traffic without prior knowledge of the website the user is visiting. Furthermore, our defense is a lightweight solution that introduces only a moderate bandwidth overhead without incurring latency overhead. The practical, effective, and lightweight nature of our defense allows it to be adopted by anonymous systems such as Tor.

We summarize the contributions of this paper as follows:

- We propose PEZD, a practical and effective lightweight defense. It breaks distinctive traffic patterns of the website by injecting different distributions of dummy packets for each load. PEZD is a website-agnostic defense that does not require prior knowledge of the website the user is visiting, making the defense more practical. Furthermore, PEZD is a lightweight defense that incurs only moderate bandwidth overhead and does not need to delay packets.
- We conduct extensive experiments to evaluate PEZD, and the results show that it outperforms the previous WF defense, WTF-PAD [17], FRONT [27], and Surakav [30]. In the closed-world setting, it reduces the accuracy of the state-of-the-art attack, Var-CNN, from 97.56 % to 19.39 %, while FRONT only reduces it to 82.92 %, WTF-PAD to 93.93 %, and Surakav to 69.93 %. In the open-world setting, PEZD provides high protection performance that reduces the recall to 0.26 and the precision to 0.54.
- We show that PEZD still provides high defense performance even if dummy packets are injected only on the client-side. In addition, we also show that PEZD does not require significant effort to maintain during deployment.

The rest of the paper is organized as follows. Section 2 provides a review of related work, then Section 3 outlines the threat model. Our proposed defense, PEZD, is presented in Section 4, and its evaluation is in Section 5. We discuss related issues of PEZD in Section 6 and summarize the paper in Section 7.

2. Related work

2.1. WF attacks

The WF attack is a classification problem in which an attacker collects traffic traces from targeted websites as instances to train a classifier for identifying unknown traces. Existing WF attacks are broadly divided into traditional machine learning with hand-crafted features and deep neural networks (DNNs) with automatically extracted features. **Traditional machine learning with hand-crafted features:** WF attacks based on traditional machine learning needs hand-crafted features, using Naïve-Bayes [12], random decision forests [7], k-Nearest Neighbour [6], and Support Vector Machines (SVM) [4,5,8] as classifiers. We introduce a well-known defense of this type.

CUMUL [8]: Panchenko et al. proposed a novel feature representation method, using the accumulated sum of packet sizes as an abstract representation, and sampling the accumulated sum of packet sizes by piecewise linear interpolation to obtain additional features. They achieved over 91 % accuracy in the closed-world setting using the SVM classifier.

DNN with automatically extracted features: Deep neural networks have been widely applied to WF attacks. The DNN-based attacks can automatically learn features from raw traffic traces.

AWF [11]: Rimmer et al. evaluated the performance of Stacked Denoising Auto Encoder (SDAE), Convolutional Neural Network (CNN), and Long Short-Term Memory (LSTM) models. The CNN among models performed best, with an accuracy of over 96 % in the closed-world setting.

DF [9]: Sirinam et al. designed a better attack model. They achieved 98.3 % accuracy in the closed-world setting and also achieved 0.95 TPR and 0.007 FPR in the open-world setting. In addition, they also evaluated the effectiveness of the attack against WF defenses, significantly undermining WTF-PAD, achieving over 90 % accuracy in the closed-world setting.

Var-CNN [10]: Var-CNN used ResNet, the state-of-the-art CNN architecture in computer vision proposed by He et al. [37], achieving 98.8 % accuracy in the closed-world setting. In the open-world setting, Var-CNN has a 0.98 Multi-TPR and a 0.003 FPR.

2.2. WF defenses

WF defenses inject dummy packets and/or delay the original packets according to different strategies to prevent attackers from identifying the target website, which introduces bandwidth and latency overheads. Existing WF defenses can be broadly divided into two classes: website-specific defenses and website-agnostic defenses. We summarize the representative defenses in Table 1 in terms of bandwidth overhead, defense assumptions, and defense performance.

Website-specific defenses: These defenses require knowledge of the traffic patterns of a specific website to generate defense information, and accordingly, they assume that the defender has prior knowledge of which websites the user visits to collect traffic patterns for those websites.

In early WF defenses, some regularization-based defenses were proposed that aim to fit traces into pre-defined patterns. Glove [14] and Supersequence [6] cluster the websites by their features and compute a uniform sending pattern for each group. When the websites that users visit are in a group, it will load the uniform pattern of the group so that the websites in the group generate a similar traffic trace. W-T [28] confuses the classifier by making the traffic trace appear like another website's traffic trace through burst molding. To easily moldable burst sequences, they modify the browser to half-duplex mode.

Many WF defenses utilizing adversarial perturbations have been proposed against WF attacks [18,22–25,29]. This type of defense exploits the weaknesses of DNN models that can cause the misclassification of classifiers even using small perturbations. Mockingbird [18] and WF-GAN [25] generate adversarial perturbations requiring the entire traffic trace, which is unrealistic under the real scenario. Whereas the network is in real-time, when the entire traffic sequence is captured, then the user session has concluded. Nasr et al. [29]. proposed Blind that uses GAN to address the limitation of protecting user traffic in real-time. Shan et al. [23] proposed Dolos, which leverages adversarial patches to resist WF attacks. Dolos can inject adversarial patches anywhere in the real traffic since the adversarial patches have the location-agnostic property. Qiao et al. proposed Acup3 [34], a novel approach that employs a many-to-one website imitation strategy to increase the similarity of traffic patterns across different websites. Moreover, Gu et al. [38] employed the Grad-CAM algorithm to identify key segments for classification and optimized adversarial patch generation through segmentation and restriction values. Although these defenses have low bandwidth overhead and high protection performance, some limitations hinder their implementation:

- These defenses ignore the fact that Tor provides defenses for all websites, and users can access any website using Tor. These defenses can only protect a few websites and lack defense generality.

Table 1

Comparison of representative WF defenses in terms of overhead, protection performance, and defense assumptions. Overhead definition: None is no overhead, Low is less than 30 % overhead, Medium is about 30–70 %, High is about 70–100 %, and Very High is more than 100 %. We define a defense to be defeated is that an attack achieved with over 80 % accuracy in the closed-world setting.

Category	Defense	Bandwidth overhead	Latency overhead	Defense assumptions	Defeated by an attack
Website-specific	Glove [14]	High	None	Prior knowledge of websites	No
	Supersequence [6]	Very High	High	Prior knowledge of websites	No
	W-T [28]	Medium	Medium	Prior knowledge of websites	No
	Mockingbird [18]	Medium	None	Prior knowledge of traffic traces	No
	WF-GAN [25]	Low	None	Prior knowledge of traffic traces	No
	Blind [29]	Low	None	Prior knowledge of websites	Yes [33]
	Dolos [23]	Low	None	Prior knowledge of websites	No
Website-agnostic	Acup3 [34]	Low	None	Prior knowledge of websites	No
	BuFLO [19]	Very High	Very High	None	No
	Tamaraw [20]	High	Very High	None	No
	Cs-BuFLO [21]	Very High	Very High	None	No
	Decoy [26]	High	None	None	No
	WTF-PAD [17]	Medium	None	None	Yes [9]
	FRONT [27]	Medium	None	None	Yes
	FRONT-U [35]	Medium	None	None	Yes
	Surakav [30]	Medium	Low	None	No
	AAP [36]	Medium	Low	None	Yes
	TBP [32]	Medium	Low	None	No
	PEZD (our work)	Medium	None	None	No

- Even if only a few censored websites are protected, the websites may vary across countries and regions, which makes these defenses hard to implement.
- Due to frequent website content updates or Tor Browser Bundle (TBB) changes, the website patterns may change. As a result, these website-specific defenses may need to periodically update the defense information to ensure their effectiveness, which consumes massive computing resources.
- In addition, some of these defenses [24,29] lack the capability to withstand non-DNN WF attacks that use statistical features such as packet counts and connection duration.

Website-agnostic defenses: Website-agnostic defenses do not rely on specific traffic trace patterns of the website, and so these defenses do not require prior knowledge of websites users visit.

BuFLO [21], Tamaraw [22], and Cs-BuFLO [23] transmit at a constant rate to mask traffic features, which incur bandwidth and latency overhead several times. Panchenko et al. [26] proposed Decoy, which randomly loads a decoy website whenever a user visits a website to obfuscate the classifier, which requires about 100 % of the bandwidth overhead.

To mitigate the high overhead, researchers have proposed manifold lightweight defenses that aim to strike a balance between defense performance and overhead, such as FRONT [27], WTF-PAD [17], and Surakav [30]. WTF-PAD is a kind of Adaptive Padding (AP) [39] algorithm. WTF-PAD obtains a time t by sampling the time of the loaded website. When the countdown of t ends, a dummy network packet is sent. WTF-PAD aims to hide the excessive time interval between network packets in the traffic. Moreover, Yang et al. [36] find that rapidly consecutive requests and responses within a short time interval have a significant impact on traffic patterns. In response, they proposed an advanced adaptive padding (AAP) aimed at modifying the burst nature of website traffic. However, WTF-PAD and AAP have been undermined by the state-of-the-art DF attack, achieving up to 80 % accuracy in the closed-world setting. Gong and Wang [27] find that WF attacks depend on feature-rich trace fronts. Therefore, they proposed FRONT, which injects dummy packets according to Rayleigh distribution to obfuscate the features of traffic trace fronts. However, FRONT provides insufficient protection for the latter part of traffic traces. We find that attackers can still identify the defended traffic traces by exploiting the features present in the latter part of the traces. Moreover, Attarian et

al. [35] find that CNNs focus more on the transitions between uploads and downloads. Consequently, they proposed an extension of FRONT, called FRONT-U, which performs padding only on the client side to reduce overhead. In addition, Surakav [30] uses Generative Adversarial Networks (GANs) to generate diverse reference traces. The sending patterns of real packets follow these traces, ensuring different traffic traces for each user visit. A significant drawback of employing GAN to generate reference traces is the substantial computational overhead. This significant computational burden may impede the practical deployment of this approach, especially in resource-constrained devices, such as when users use access Tor via mobile devices.¹ Moreover, Surakav requires to delay packets, which engenders potential memory shortage problems for Tor's relays [40]. Wang et al. [32] proposed a novel tree structure to describe the website browsing process. Based on this, they introduced a Tree structure Burst-sequence Padding (TBP), which determines dummy burst sequences through multiple random distributions. However, multiple determined distributions did not effectively disrupt WF attacks, as Var-CNN can achieve 80 % accuracy.

Other defenses: Recently, some WF defenses based on traffic splitting have been proposed [41–43], where they split traffic over several sub-circuits in a random manner, which is slightly different from our threat model. Therefore, we do not discuss these methods in this paper.

Overall, website-specific defenses have impractical assumptions that hinder implementation, while website-agnostic defenses either incur high overhead, cannot provide high protection performance, or introduce latency overhead.

3. Threat model

3.1. Attack model

We consider the same threat model as existing WF attacks [4,6–11, 17,18,23,27]:

- WF attacks assume an attacker, located between the user and the Tor entry node, sniffs users' traffic, and the attack model is shown in Fig. 1. The attacker is passive, meaning they cannot drop, delay, or modify network packets.

¹ Tor Browser for Android has surpassed 10 million downloads on the Google Play Store.

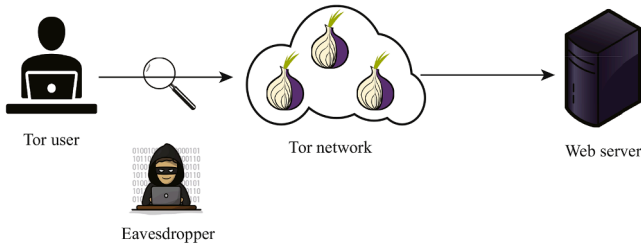


Fig. 1. The WF attack model.

- We consider the attacker has full knowledge of the defense method implemented by Tor [6–11,17,18,23,27], which means the attacker can collect the defended traffic to train a classifier. We assume that the attacker's classifier is trained using already defended traffic. Given that Tor is an open-source project, it is reasonable to assume that potential attackers would have access to the defended traffic traces and could use them to train more robust attack models.
- We consider WF in closed-world and open-world settings [8–11,17,27]. In the closed-world setting, users visit only a subset of websites trained by attackers, called monitored websites. However, this setting was criticized by Juarez et al. [44] as unrealistic since users can visit any website. The open-world setting is more realistic, and users can visit unmonitored websites. It is a binary classification setting. The attacker uses the trained classifier to distinguish whether the website the user visits is monitored or unmonitored.
- We consider the attacker to use only packet direction sequences as input to the model. This assumption is the same as many previous attacks [9–11] and defenses [18,22–24,28,44,45]. Tor transmits data in fixed-size (512-byte) packets called cells. A sequence of packets collected throughout the website load process constitutes a traffic trace. For example, the traffic trace of a website containing only packet directions can be denoted as $T_p = [O_1^p, O_2^p, I_3^p, I_4^p, I_5^p, I_6^p, O_7^p, \dots, I_n^p]$, where O_i^p is the i -th packet in the traffic trace, i is the location of the packet, n represents the number of packets, O represents the packet is an outgoing packet, I means the packet is an incoming packet, and p indicates classes of the website. In a traffic trace, outgoing packets are represented as 1, and incoming packets are represented as -1. Therefore, the input used by the attacker is $T_p = [1, 1, -1, -1, -1, -1, 1, \dots, 1]$.

3.2. Defense model

We assume that the defender has the following capabilities:

- The defender can inject dummy network packets in real traffic but not discard, modify, or encrypt users' network packets [4,6,14–28]. Defenders can inject bidirectional dummy packets (outgoing and incoming) with the cooperation of both the Tor client and a node in the Tor circuit. We consider the node in the Tor circuit to be the Tor bridge.²
- The defender has no prior knowledge of the website the user is visiting [17,19–21,26,27], which means that the defender cannot collect traffic traces of the websites the user will visit in advance.

4. PEZD

In this section, we first introduce the idea behind the PEZD in Section 4.1. In Section 4.2, we describe PEZD in detail.

² Tor bridges are commonly used to evade censorship, and they are often used by WF defenses to generate dummy incoming packets to reshape traffic [17,18,23,27]

4.1. Motivation and intuition

We believe that defense meets deployability and should provide high protection performance and defense generality while imposing only moderate bandwidth overhead and no latency overhead. High protection performance means that the defense can resist state-of-the-art WF attacks, and defense generality means that the defense can protect all websites. A moderate bandwidth overhead indicates that a moderate number of dummy packets are injected into the traffic. No latency overhead means that real packets are sent without delay.

FRONT provides low protection against state-of-the-art WF attacks, despite its defense generality, moderate bandwidth overhead, and zero-delay. To have a deep understanding of the reasons why FRONT provides low protection performance, we introduce an explainable artificial intelligence (XAI) framework to analyze it. An XAI framework is a tool that generates reports on the model's operation and makes an attempt to explain how it operates [46]. The XAI framework we use here is Locally Interpretable Model Agnostic Interpretation (LIME) [47]. LIME operates by generating perturbed samples around a target example and fitting these samples with an interpretable linear model. This technique enables us to identify the key features driving the model's predictions, effectively making any classifier's decision-making process locally interpretable, regardless of its underlying complexity. The interpretation process is shown in Eq. (1),

$$\xi(x_i) = \arg \min_{g \in G} L(f, g, \pi_x) + \Omega(g) \quad (1)$$

where $\xi(x_i)$ is the interpretable feature explained by LIME, $L(f, g, \pi_x)$ is loss function, f is the black-box model, g is the explainer, π_x is the similarity measure between data points x , and $\Omega(g)$ is the penalty for the complexity model g . LIME generates a list of explanations, with each explanation indicating the role of each feature in the forecast of a particular data sample [47].

We use LIME to analyze which packets in undefended traffic and FRONT play a more important role in WF attack models. We train the DF attack model on the DS-19 dataset (See Section 5 for more detail). Typically, for WF attacks based on deep learning models, each packet (denoted as 1 or -1) in a traffic trace serves as a feature. We randomly pick four classes (indexing 8, 21, 86, and 94 in the DS-19 dataset) with 100% attack accuracy to analyze their features since we want to analyze why FRONT fails. Note that due to the model's fixed input, traffic with less than 10,000 packets is padded with zeros. However, LIME also generates an importance score. For intuition, we only show the importance level of the real packets. Specifically, we conducted a statistical analysis of the locations of the 1000 most important packets across 10 traffic traces per class, for both the FRONT and undefended datasets. The traffic trace is segmented into a series of intervals, with each interval having a fixed length of 500 packets. We illustrate the number of significant packets within each interval, as shown in Fig. 2.

For undefended traffic, we observe that the most important packets are mostly distributed within the first interval of 500, suggesting that the WF attack heavily relies on packets in the initial portion of traffic traces. In contrast, for FRONT, we find that the model relies more on the features towards the rear of the traffic. In other words, the model can learn distinctive patterns in the latter part of the traffic. This is due to the fact that FRONT adds more dummy packets to the front of the traffic and fewer packets are added to the back, resulting in an incapability to robustly interfere with the back patterns of the traffic traces. In light of the powerful learning capabilities of deep learning models, even if there are intra-website traces with a small number of shared patterns, they can still be learned. The analysis of the reasons for FRONT's failure also sheds new light on the design of effective defenses, where the distribution of dummy packets should be different for each traffic trace. More specifically, to avoid the common patterns in intra-website traffic traces, the distribution of dummy packets for each traffic trace should be distinct.

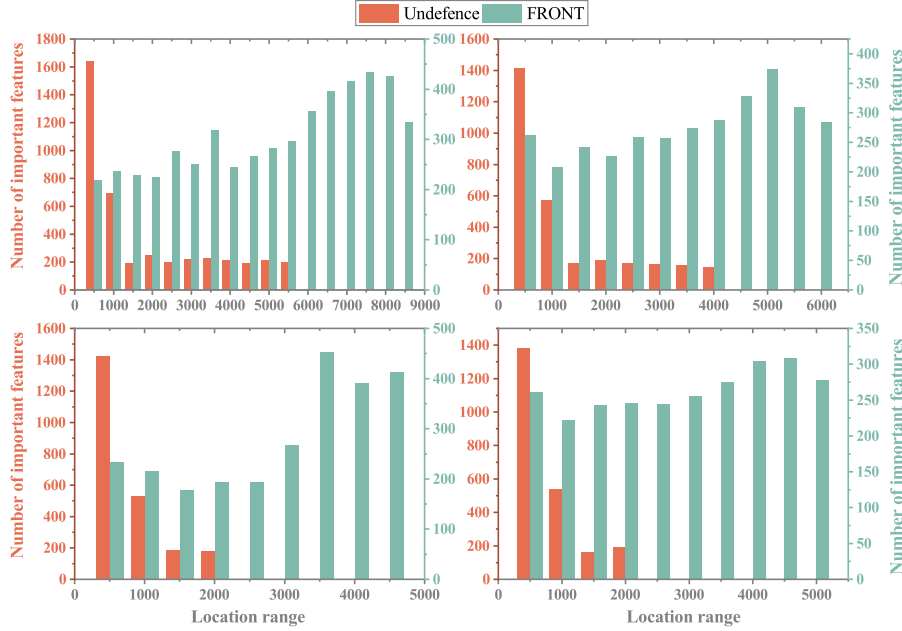


Fig. 2. Visualization of the number of important features within intervals. The number of feature importance indicates how many significant features fall within an interval, with each interval having a length of 500.

Next, we describe how to construct a distribution of dummy packets. Intuitively, a probabilistic approach seems feasible. However, previous research has shown that constructing effective probability distributions for this purpose faces significant challenges. For instance, previous defense methods, such as FRONT, attempted to increase diversity by employing various probabilistic parameters in the generation of dummy packets. Nevertheless, Var-CNN demonstrated superior performance, achieving an accuracy of over 80%. Even sophisticated techniques like TBP, which combined multiple probability distributions to generate burst distributions, still remained fragile against attacks. Var-CNN, for example, achieved an accuracy of over 70% against TBP. A critical reason for this vulnerability is the use of deterministic distributions to generate dummy packets. In these approaches, the distribution of dummy packets for each traffic trace is derived from one or a few fixed distributions. This increases the similarity of dummy packet distributions across different traffic traces. An ideal scenario would involve generating dummy packet distributions from different distributions for each traffic trace, thereby significantly enhancing the diversity between traffic traces. In addition, Surakav uses a GAN to generate diverse reference traces. However, using a GAN significantly increases computational overhead. This motivates us to seek a computationally efficient way to generate diverse distributions of dummy packets for each traffic trace.

We observe that traffic inherently contains diverse distributions, which inspired us to leverage these natural patterns directly. Each website has unique patterns, including unique packet ordering, packet directions, and packet timing, which makes the traffic traces of different websites significantly different, which is also why the WF attacks are successful. We observe significant differences in the distribution and number of outgoing packets in traffic traces of different websites. We use Kernel Density Estimation (KDE), a nonparametric model used for estimating probability distributions, to estimate such a distribution from a website. The KDE is shown in Eq. (2),

$$\hat{f}_h(x) = \frac{1}{n} \sum_{i=1}^n K_h(x - x_i) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right) \quad (2)$$

where x_i is identically distributed samples, and $K(\cdot)$ is a kernel function, which specifies how to compute the probability density given the dis-

tance, and h is a smoothing bandwidth, which controls how smooth the estimated density curve is.

Fig. 3 shows a visualization of the distribution of outgoing packets of websites (website indexing 32, 39, 50, 55, 78, and 92 in the dataset), the dataset provided by Gong and Wang [27]. We observe significant variations in the distribution of outgoing packets across various websites. Therefore, We can exploit the distribution of outgoing packets from different websites to randomize the distribution of dummy packets. A greater divergence in the distribution of outgoing packets leads to greater variations in the distribution of dummy packets across different traffic traces of the same website. This makes it difficult for intra-website traffic traces to have common patterns that can be learned by an attacker.

4.2. Defense design

In this section, we describe in detail how to exploit differences in the location of outgoing packets from different websites to obfuscate traffic traces.

We add dummy packets according to the outgoing packet location information extracted from traffic traces, and a visual representation of location information is shown in Fig. 4. Therefore, we are required to collect a traffic trace pool including many traffic traces. Note that the traffic traces in the traffic trace pool can be any website. The difference in websites we utilize does not care which website it is. Therefore, PEZD does not require any prior information for the websites users visit. We describe the setting of the traffic trace pool in detail in Section 5.3.

Next, we describe adding dummy packets according to the extracted location information. For simplicity, we do not distinguish the adding dummy packets' direction (outgoing and incoming). Suppose we need to protect a website w . We randomly select a traffic trace from the traffic trace pool, which is called the non-source traffic trace. The location information of all outgoing packets in the non-source traffic trace is counted. Afterward, we add dummy packets in real traffic at the same location as the outgoing packets of the non-source traffic trace. The same location means that the outgoing packet is at the k -th in the non-source traffic trace, and the inject dummy packet is also at the k -th in the real traffic. Note that we ignore the location of the first packet from the non-source traffic trace since the client needs to

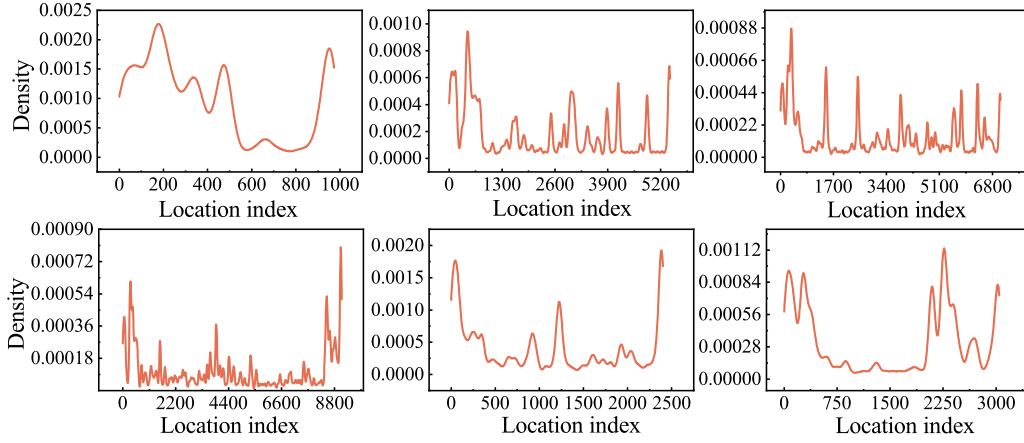


Fig. 3. Probability density function of outgoing packets from different websites.

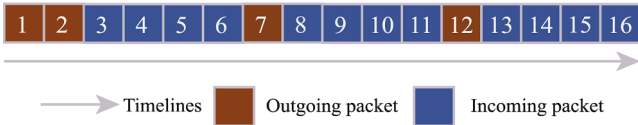


Fig. 4. A visual representation of location of packets. The number indicates the location of the packet.

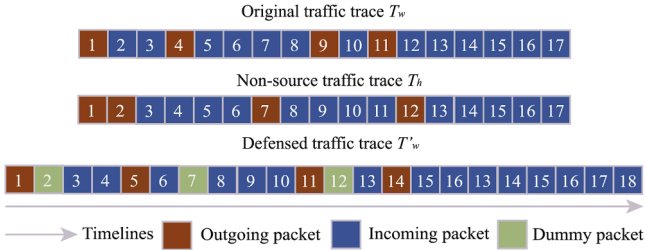


Fig. 5. Padding process of dummy packets.

send the first packet to establish a connection with the server, and the first packet cannot be a dummy packet. For example, we randomly select a non-source traffic trace $T_h = [O_1^h, O_2^h, I_3^h, I_4^h, I_5^h, I_6^h, O_7^h, I_8^h \dots I_b^h]$, where h can be any website. We extract the location information of outgoing packets of T_h denoted as $L_h = \{2, 7 \dots\}$. We assume a traffic trace of the website w is $T_w = [O_1^w, I_2^w, I_3^w, O_4^w, I_5^w, I_6^w, \dots I_m^w]$. We inject dummy packets to T_w according to L_h to get $T_w' = [O_1^w, D_2, I_{2 \rightarrow 3}^w, I_{3 \rightarrow 4}^w, I_{4 \rightarrow 5}^w, D_7, O_{6 \rightarrow 8}^w, I_{7 \rightarrow 9}^w, I_{8 \rightarrow 10}^w, \dots I_{m \rightarrow m+u}^w]$, where $i \rightarrow j$ means that a packet was i -th in the traffic trace, and after adding dummy packets is the j -th, and D represents the packet is a dummy

packet, and u is the total number of dummy packets added. Fig. 5 shows a padding process.

Since the number of outgoing packets accounts for a small proportion (about 10%) of a traffic trace, adding such a number of dummy packets does not significantly impact real traffic patterns. Therefore, we select multiple non-source traffic traces from the traffic trace pool to extract the location information of outgoing packets. Specifically, We set a number K to determine the number of non-source traffic traces. A larger K means more non-source traffic traces are selected to extract the location information of outgoing packets, which means more bandwidth overhead.

In addition, we randomly offset the extracted location information to avoid the concentration of dummy packets in the front. Suppose we select non-source traffic traces with fewer packets, which means smaller location information for outgoing packets. The majority of the dummy packets will be focused in front of real traffic, and none will be sent to the back, as shown in Fig. 6. Therefore, we offset a random value for each location information extracted from the traffic traces to address the possible imbalance in the distribution of dummy packet locations, as shown in Fig. 7.

In the above description, we do not distinguish the direction of the added dummy packets. In real networks, dummy packets can be injected bidirectionally. We set the parameters K_c and K_s to determine the number of non-source websites selected by the Tor client and Tor bridge, respectively. The bandwidth overhead is determined by $K_c + K_s$, as detailed in Section 5.3. The location information extracted from K_c non-source traffic traces is placed in the client-side location information set L_c , and the location information extracted from K_s non-source traffic traces is placed in the Tor bridge-side location information set L_s .

Note that PEZD does not need to know the entire traffic trace. The Tor client or bridge only requires to know the location information of dummy packets to be sent. PEZD is very easy to implement and implements the state machine shown in Fig. 8. For simplicity, we only

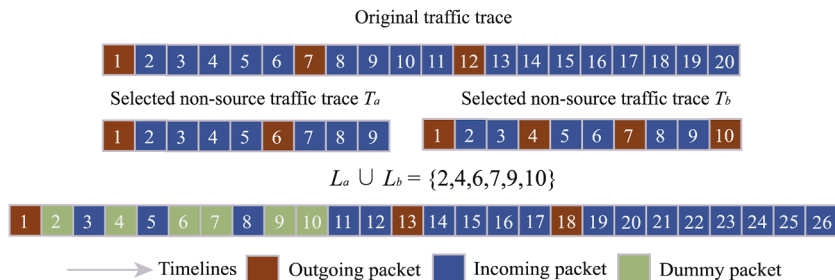


Fig. 6. The result of padding according to location information of multiple traces. L is the set of dummy packet location information.

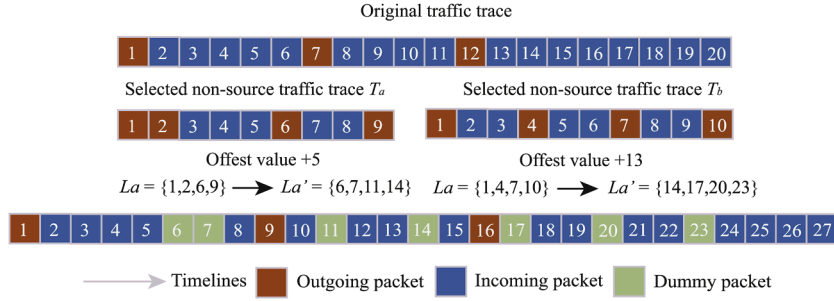


Fig. 7. Location information offset and padding process.

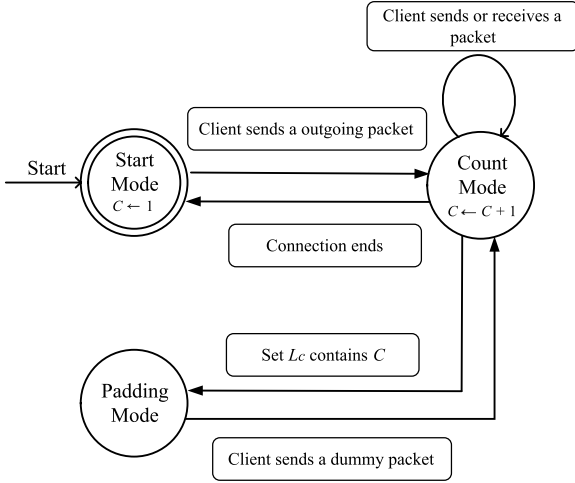


Fig. 8. The finite state machine of PEZD implementation on client-side.

describe the client-side state machine, and the Tor bridge operations are symmetric.

Start Mode. Starting in Start Mode, the client has obtained the location information set L_c and sets counter C to 1. The defense waits for the client to send an outgoing packet to establish a connection with the server. When the client sends a real packet, switch to Count Mode.

Count Mode. In Count Mode, when the client receives or sends a packet, counter C adds 1. The defense relies on counter C to decide when to send a dummy packet. When set L_c contains counter C , switch to Padding Mode.

Padding Mode. In Padding Mode, the client injects a dummy packet into the traffic sequence. After padding, the defense returns to Count Mode to continue counting the number of packets. When the connection ends, switch to Start Mode, and the defense ends.

5. Evaluation

5.1. Dataset

DS-19 [27]: This dataset was collected by Gong and Wang to evaluate FRONT and was collected between February to April 2019, utilizing Tor Browser 8.5a7 running on Tor 0.4.0.1-alpha. Data collection was automated through command-line interactions with the Tor Browser. They collected Alexa [48] top 100 websites as monitored websites with 100 traffic traces each, which means there are 100 classes (each website represents a class) with 100 samples per class. Additionally, they collected 10,000 other websites from Alexa as unmonitored websites with one traffic trace each. These unmonitored websites were used to evaluate the performance in an open-world scenario. Each traffic trace in the dataset consists of a sequence of packet features, where each packet is

represented as a tuple $f_k = \langle t_k, d_k \rangle$, with t_k being the packet timestamp and d_k indicating the packet direction.

DS-14 [6]: This dataset was collected by Wang in 2014 on Tor Browser 3.5.1. It contains 100 websites as monitored websites with 90 traffic traces each, these websites were selected from sites blocked in China, the UK, and Saudi Arabia. It also collected 9,000 websites as unmonitored websites with one traffic trace each. The traffic traces in this dataset follow the same feature representation as DS-19.

We use the DS-19 and DS-14 datasets because they provide timestamps and directions of packets that can be used to simulate FRONT and WTF-PAD defense. We follow Gong and Wang's settings [27] to set the length of the traffic trace length to 10,000 less than 10,000 filled with 0.

5.2. Evaluation metrics

In the closed-world setting, we use accuracy as the evaluation metric. In the open-world setting, we use true positive rate (TPR), false positive rate (FPR), precision, and recall as evaluation metrics. In particular, we consider if it is classified as monitored with a probability above the pre-defined threshold, otherwise, a false negative (FN). A false positive (FP) is an unmonitored traffic trace incorrectly classified as monitored with a probability above the threshold, otherwise, a true negative (TN). The bandwidth overhead is defined as the ratio of the total number of padded dummy packets to the number of real packets in the original traffic sequence. Latency overhead is defined as the ratio of the delay time incurred by real packets to their original transmission time. For statistical soundness, we conduct the experiment 10 times and computed the mean value as the final result.

5.3. Parameters setting

The size of the traffic traces pool: PEZD requires selecting non-source websites from the traffic trace pool to extract location information. In our experiments, we set the websites in the traffic trace pool not to overlap with the defended websites, which is a more realistic scenario since PEZD does not have prior knowledge of the websites visited by users. In the closed-world setting, we use 10,000 unmonitored websites as the traffic trace pool. In the open-world setting, we select 2000 unmonitored websites as the traffic trace pool. We evaluate the impact of the size of the traffic trace pool on the WF attack performance in Section 5.4.

The range of random offset values: PEZD needs random offsetting of dummy packet locations to address the concentration of dummy packets in the front. To set the appropriate random offset range, we counted the number of packets in traffic traces. More than 90 % of traffic traces have fewer than 10,000 packets, and more than 70 % of traffic traces have fewer than 6000 packets. Therefore, we set the offset value in the range $[-10000, 10000]$. Note that this is the randomly selected offset values range, not the range of dummy packets added to the traffic trace.

The setting of K : In the PEZD design, the parameter K represents the total number of non-source traffic traces selected from the traffic trace

Table 2
The defense parameters in DS-19 and DS-14 datasets.

Defense	Parameters	Dataset
WTF-PAD	<i>normal_rcv</i>	DS-19 DS-14
FRONT	$N_c = N_s = 2500, W_{\min} = 1s, W_{\max} = 14s$	DS-19 DS-14
Tamaraw	$L = 100, \rho_{out} = 0.04, \rho_{min} = 0.012$	DS-19 DS-14
Surakav	$\delta = 0.2, \rho = 40ms$	DS-19 DS-14
FRONT-U	$N_c = 5000, N_s = 0, W_{\min} = 1s, W_{\max} = 14s$	DS-19 DS-14
AAP	$N = 4000, Q = 0.4$	DS-19 DS-14
TBP	$P = 5000, L_{max} = 20, L_{min} = 2$	DS-19 DS-14
PEZD	$K_c = 9, K_s = 2$	DS-19
	$K_c = 12, K_s = 3$	DS-14

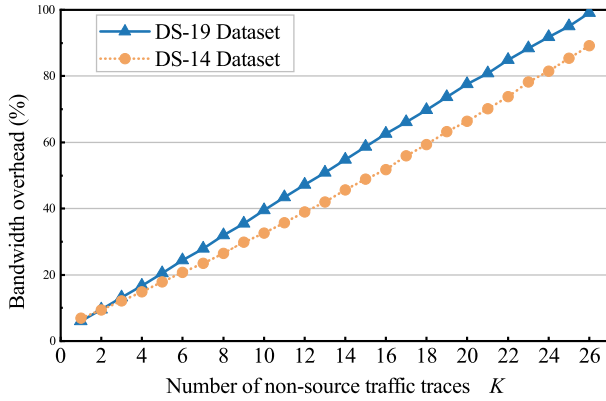


Fig. 9. The bandwidth overhead of DS-19 and DS-14 datasets for the selected numbers of non-source traffic trace K .

pool, which determines the bandwidth overhead of PEZD. A larger K indicates a larger number of selected non-source traces. Since dummy packets are injected based on the location of the outgoing packets in the non-source traffic traces, this incurs higher bandwidth overhead. Fig. 9 shows the bandwidth overhead corresponding to the number of K for DS-19 and DS-14 datasets. We observe that the bandwidth overhead gradually increases as K increases. The bandwidth overhead is less than 30% when K is less than 7 and greater than 70% when K is greater than 17. In general, PEZD has a moderate bandwidth overhead when K is in the interval [7, 17]. We try to compare the performance of various defenses for the same bandwidth overhead (WTF-PAD and FRONT claim a bandwidth overhead of about 40%), and we thus set $K = 11$ in the DS-19 dataset (43% bandwidth overhead) and $K = 15$ in the DS-14 dataset (48% bandwidth overhead). In addition, We define K_c and K_s to represent the number of traffic traces selected from the traffic trace pool by the client and the server, respectively. We discuss the effect of the K_c and K_s parameters on performance in Section 5.4.

Defense parameters setting: For the parameter setting of WTF-PAD, FRONT, and Tamaraw, we follow the default experimental parameters in their papers. To ensure a fair comparison, we tuned the defense parameters for Surakav, FRONT-U, AAP, and TBP on the experiment dataset to achieve a low latency overhead while maintaining a comparable bandwidth overhead to our defense. We summarize the defense parameters in Table 2.

5.4. Closed-world evaluation

In this section, we evaluate the effectiveness of PEZD in the closed-world setting. First, we make a comprehensive comparison of PEZD with state-of-the-art defenses. Then, we evaluate the performance of PEZD in

Table 3
Accuracy and overhead of WF defense against attack in the closed-world scenario.

Dataset	Defenses	Overhead		Var-CNN	DF	CUMUL
		Latency	Bandwidth			
DS-19	No defense	0	0	97.56 %	96.36 %	95.96 %
	WTF-PAD	0	29.97 %	93.93 %	93.07 %	70.86 %
	FRONT	0	44.01 %	82.92 %	68.0 %	30.73 %
	Tamaraw	171.15 %	71.05 %	16.14 %	10.22 %	16.54 %
	Random	0	47.3 %	94.53 %	94.99 %	93.38 %
	Surakav	12.1 %	44.61 %	69.93 %	70.0 %	48.77 %
	FRONT-U	0	43.72 %	91.55 %	79.82 %	62.88 %
	AAP	0	50.94 %	80.26 %	86.78 %	53.82 %
	TBP	0	45.39 %	70.7 %	65.34 %	35.75 %
	PEZD	0	43.39 %	19.39 %	37.88 %	21.45 %
DS-14	No defense	0	0	81.53 %	92.18 %	91.17 %
	WTF-PAD	0	47.69 %	69.76 %	79.20 %	58.03 %
	FRONT	0	69.20 %	61.26 %	60.48 %	25.12 %
	Tamaraw	78.31 %	92.13 %	12.55 %	16.46 %	13.81 %
	Random	0	47.32 %	83.75 %	86.81 %	83.65 %
	Surakav	5.1 %	76.96 %	42.98 %	54.28 %	37.18 %
	FRONT-U	0	52.26 %	84.46 %	76.46 %	61.77 %
	AAP	0	45.52 %	61.37 %	65.68 %	38.91 %
	TBP	0	44.74 %	56.07 %	56.94 %	39.68 %
	PEZD	0	48.73 %	21.51 %	33.20 %	29.70 %

injecting dummy packets only on the client-side or server-side. Next, we evaluate the impact of the size of the traffic trace pool on the protection performance of PEZD. Finally, We evaluate the effect of the parameters K_c and K_s .

5.4.1. Evaluation against WF attacks

We evaluate the protection performance of defenses on the DS-19 and DS-14 datasets. We compare with eight representative WF defenses: WTF-PAD, FRONT, Tamaraw, Surakav, and Random. WTF-PAD, FRONT, FRONT-U, AAP, and TBP are zero-delay lightweight defenses. Surakav is a lightweight defense that incurs low latency overhead. Tamaraw is a heavyweight defense. Random is a more common randomization method that randomly adds dummy packets to traffic traces according to the uniform distribution. We consider three state-of-the-art attacks, Var-CNN, DF, and CUMUL, to evaluate these defenses. Var-CNN and DF are DNN attacks, and CUMUL is a non-DNN attack. Note that we compute the overhead for monitored websites and unmonitored websites separately for all defenses since only monitored websites are included in the closed-world setting.

Table 3 lists experiment results, showing that PEZD offers higher protection than WTF-PAD, Random, FRONT, Surakav, FRONT-U, TBP, and AAP. Specifically, Var-CNN achieves over 80% accuracy for WTF-PAD, FRONT, FRONT-U, AAP, and Random. TBP performed slightly better, but Var-CNN still attained an accuracy of over 70%. In contrast, the accuracy of PEZD was only 19.39%. Moreover, compared to Surakav, PEZD shows 40% lower attack accuracy against Var-CNN. We observe that FRONT, Surakav, AAP, and TBP only have a favorable performance for resisting the non-DNN attack. For instance, for FRONT, CUMUL only achieves 30.73% accuracy. Furthermore, we find that Random does not provide effective defense for either DNN attacks or non-DNN attacks, and WF attacks achieve over 80% accuracy. A key factor is that adding dummy packets with the same distribution to traffic from the same website does not introduce significant distinctions between them. From a distributional perspective, we posit that traffic from the same website follows a specific distribution. Since we generate dummy packets from the same distribution, the distribution of the dummy packets in different traffic is also similar, which allows the final protected traffic from the same website to still maintain a substantially similar distribution.

Tamaraw is effective against WF attacks, and it reduces Var-CNN accuracy to 16.14%. However, it incurs over 171% latency overhead and 71% bandwidth overhead, while PEZD requires only 43% of bandwidth

Table 4

Defense performance of PEZD for client-side, Tor bridge-side, and bidirectional injection of dummy packets.

Dataset	Packet direction	Bandwidth overhead	Var-CNN	DF	CUMUL
DS-19	Client-side	43.43 %	23.16 %	46.71 %	28.63 %
	Tor bridge-side	41.31 %	92.19 %	91.42 %	82.09 %
	Two-way	43.39 %	19.39 %	37.87 %	21.45 %
DS-14	Client-side	48.88 %	20.26 %	38.81 %	36.64 %
	Tor bridge-side	48.50 %	84.11 %	81.25 %	73.20 %
	Two-way	48.72 %	21.51 %	33.20 %	29.70 %

overhead to reduce Var-CNN attack accuracy to 19.39%. In addition, we find that DF is more robust to PEZD than Var-CNN and CUMUL, achieving 37.88% accuracy. Surprisingly, Var-CNN is more fragile than CUMUL for PEZD, despite its more complex structure and higher classification accuracy for other lightweight defenses.

5.4.2. Impact of dummy packet directions

The aforementioned experiments consider the injection of bidirectional dummy packets to reshape the traffic, which requires cooperation between the client and the Tor bridge. This approach inherently increases the complexity of deployment. In contrast, implementing PEZD solely on either the client-side or server-side offers easier deployment and many potential advantages. For instance, implementation only on the server-side would minimize modifications required on the client-side, and vice versa. Moreover, Client-side deployment significantly reduces the computational and bandwidth requirements for Tor bridges, particularly beneficial for overload nodes. Furthermore, the Tor bridge and client cooperation may pose privacy issues, as some users may not wish the Tor bridge to access their browsing history directly [24]. In addition, the deployment of other privacy-enhancing technologies, such as virtual private networks (VPNs), is impeded by the imperative that clients and remote endpoints cooperate to reshape traffic [49]. For client-side defense, we set the defense parameters for the DS-19 and DS-14 datasets as $K_c = 11$ and 15, respectively, with K_s set to zero for both datasets. Conversely, for server-side defense, we set the parameters as $K_s = 11$ and 15 for these two datasets, while maintaining K_c at zero.

We compare the defense performance of injecting dummy packets only on the client-side or Tor bridge-side and injecting dummy packets in both directions, and the results are shown in Table 4. PEZD defense performance does not significantly reduce for dummy packets injected only on the client-side. For instance, the accuracy of the Var-CNN attack increased by less than 4%, and the accuracy of the CUMUL attack increased by about 8% in the DS-19 dataset. The highest attack accuracy for the three attacks is 46.71% for the client-side, indicating that the client-side deployment of PEZD effectively resists WF attacks. Surprisingly, adding only incoming packets is extremely fragile and Var-CNN achieves over 90% accuracy. One possible explanation is that the model assigns lower weights to incoming packet features while still effectively identifying websites by unperturbed outgoing packet features with higher weights, consistent with findings from previous research [24]. This comparison provides strong evidence that adding dummy outgoing packets yields more effective protection.

5.4.3. Impact of traffic trace pool size on accuracy

Next, we evaluate the impact of the traffic trace pool size on the protection performance of PEZD. In the above experiment, our traffic trace pool contains 10,000 traffic traces. The fewer traffic traces in the traffic pool will have a higher chance of selecting the same non-source traffic traces, which may make the distribution of dummy packets less diverse and pose lower protection performance. To investigate the effect of this factor, we vary the number of traffic traces in the traffic trace pool from 1,000 to 10,000. The experiment is conducted on the DS-19 dataset.

The results are shown in Fig. 10. PEZD can withstand reduced traffic traces in the traffic trace pool. The defense performance of PEZD remains

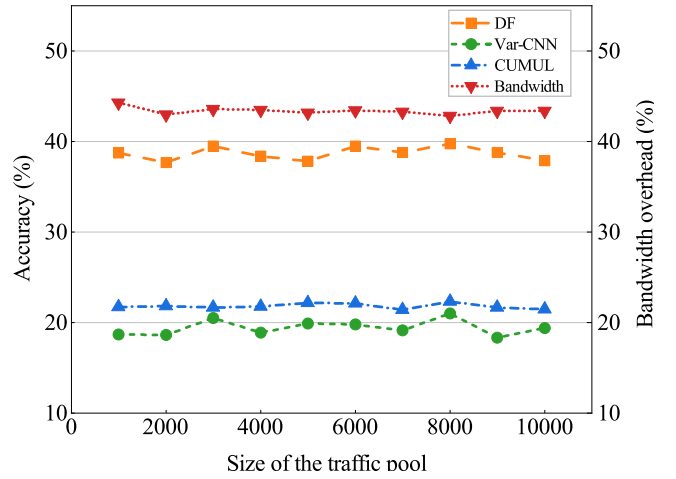


Fig. 10. Impact of the traffic trace pool size for the performance of PEZD. Three state-of-the-art attacks, DF, Var-CNN, and CUMUL, are chosen to evaluate the performance of PEZD under different sizes of the traffic trace pool.

stable even when traffic traces are reduced. Even with only 1,000 traffic traces in the traffic trace pool, the protection performance of PEZD does not degrade significantly. In more detail, the accuracy of the Var-CNN attack is 18.69% with 1000 traffic traces and 19.39% with 10,000 traffic traces. The slight change in accuracy is potentially due to the higher bandwidth overhead at 1000 traffic traces than at 10,000 traffic traces. An essential factor in the ability of our defenses to withstand the impact of reduced traffic traces is that we apply a random offset to the extracted location information. Even if different traffic traces of a website select the same non-source traffic trace, a random offset value will make the location of the dummy packets different.

5.4.4. Impact of K_c and K_s on accuracy

The parameters K_c and K_s represent the number of traffic traces selected from the traffic trace pool by the client and the server, respectively. To investigate the impact of different values of K_c and K_s on the defense performance, we adjust the quantities of K_c and K_s for a given setting of $K = 11$.

Fig. 11 shows the effect of different numbers of K_c on the defense performance for given $K = 11$ in the DS-19 dataset. CUMUL, Var-CNN, and DF have the lowest attack accuracy for K_c values of 6, 8, and 9, respectively, resulting in accuracies of 18.61%, 19.05%, and 37.88%. The percentage of dummy outgoing packet overhead to total dummy packets bandwidth overhead is 61.5%, 77.8%, and 85.6% for K_c values of 6, 8, and 9, respectively. We define $\alpha = K_c / (K_c + K_s)$, which is the proportion of non-source traces selected by the client-side. PEZD has an acceptable defense performance for the three attacks when K_c is set to 9. The optimum α value is around 0.8. Therefore, we recommend maintaining this value of α in the defense for optimal performance.

5.5. Open-world evaluation

We compare the performance of defenses in the more realistic open-world setting. We use the DF attack to evaluate defenses in the DS-19 dataset since DF has better performance for defenses. As we have shown the effectiveness and high overhead of Tamaraw in the closed-world setting, we exclude it here and focus comparisons on lightweight defenses. We follow the previous hypothesis to include unmonitored websites in the training dataset, known as the standard model [6–9]. Following the DF attack, we use prediction probabilities to distinguish between monitored and unmonitored websites. A traffic trace is considered a true positive if it is classified as a monitored website and its maximum prediction probability exceeds the threshold. Otherwise, it is classified as an unmonitored website. To ensure comprehensive evaluation, we

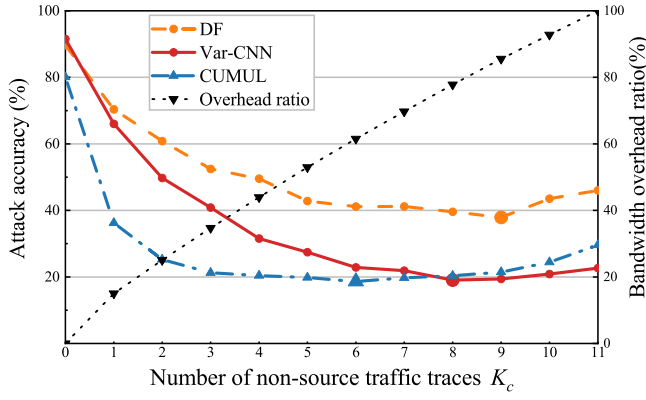


Fig. 11. The effect of different numbers of K_c on the defense performance for given $K = 11$. The overhead ratio is the ratio of dummy outgoing packages to total bandwidth overhead. The optimal number of K_c is indicated by a larger symbol in each subplot.

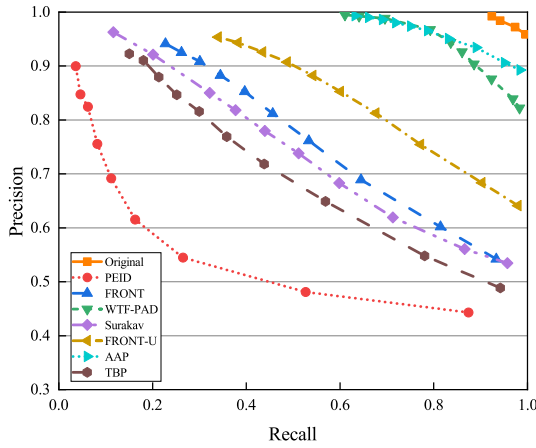


Fig. 12. Open-world scenario: Precision-Recall curve.

implement multiple threshold values in line with the DF attack. In our open-world evaluation, the training set contains 13,000 traffic traces (9000 from monitored websites and 4000 from unmonitored websites), and the testing set includes 5000 traffic traces (1000 from monitored websites and 4000 from unmonitored websites). The test set contains more unmonitored websites because the size of the set of unmonitored websites is usually larger than the set of monitored websites, which is more in line with the real situation [9,17]. For PEZD, the traffic trace pool contains only the remaining 2000 traffic traces from unmonitored websites. For unmonitored websites in the open-world setting, PEZD, FRONT, WTF-PAD, Surakav, FRONT-U, TBP, and AAP incur bandwidth overheads of 41 %, 47 %, 30 %, 49 %, 56 %, 59 %, and 53 %, respectively. Moreover, Surakav incurs 12.33 % latency overhead.

As mentioned in Section 3, in the open-world setting, the attacker distinguishes whether the website the user visits is monitored or unmonitored. If the traffic trace is identified as a monitored website, the attacker can use the multi-class classifier trained in the closed-world to predict the class of the traffic trace. We assess each defense's performance in the open-world setting by examining its TPR and FPR. Furthermore, we also use precision and recall as evaluation metrics following the recommendations of WF studies [8,9,17]. In our evaluation, we plot precision-recall curves and ROC curves.

Fig. 12 shows precision-recall curves for PEZD, FRONT, FRONT-U, WTF-PAD, Surakav, AAP, TBP, and undefended traffic. PEZD outperforms other defenses at any threshold. We observe that the recall of PEZD is smaller than FRONT, FRONT-U, AAP, TBP, and WTF-PAD for the same precision, which means that PEZD makes the attacker miss more monitored websites. Specifically, DF achieves more than 0.9 for

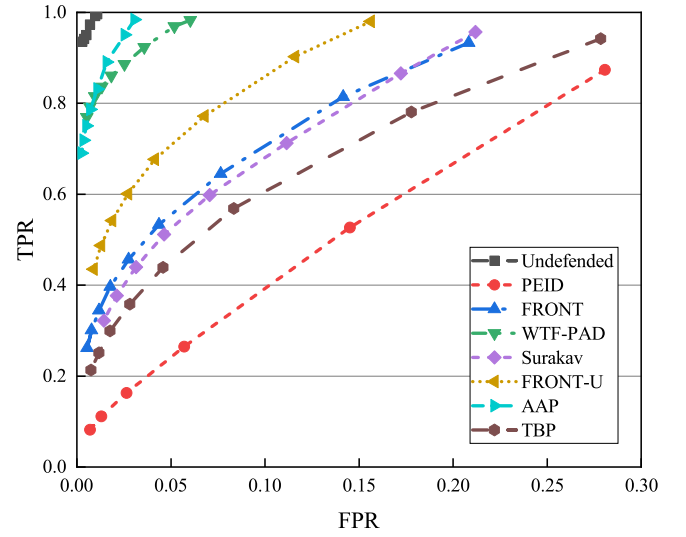


Fig. 13. Open-world scenario: ROC curve.

both precision and recall for the undefended traffic, while PEZD provides high protection performance that reduces the recall to 0.26 and the precision to 0.54. Surakav, FRONT-U, and FRONT provide weaker protection compared to PEZD, with Surakav reducing the recall to 0.51 and the precision to 0.73, FRONT reducing the recall to 0.64 and the precision to 0.81. In contrast, TBP offers slightly improved defense performance, with DF achieving a recall of 0.43 and a precision of 0.71. WTF-PAD and AAP provide lower protection performance, and DF can achieve the precision and the recall up to 0.8.

Fig. 13 shows the ROC curves for the PEZD, FRONT, WTF-PAD, Surakav, FRONT-U, AAP, TBP, and undefended traffic. PEZD provides a more robust protection performance than FRONT, Surakav, WTF-PAD, FRONT-U, AAP, and TBP, achieving lower TPR and higher FPR. At the same FPR, the TPR of PEZD is always smaller than FRONT, WTF-PAD, Surakav, FRONT-U, AAP, and TBP. For instance, at an FPR of approximately 0.07, the TPR of PEZD is 0.26, 0.23, 0.6, 0.5, 0.6, and 0.2 lower compared to FRONT, Surakav, WTF-PAD, FRONT-U, AAP, and TBP, respectively. The results show that WTF-PAD provides poor protection performance, and the DF attack achieves a TPR of 0.98 and an FPR of 0.06.

6. Discussion

6.1. Analysis

PEZD has significant advantages over the lightweight defenses WTF-PAD, FRONT, FRONT-U, AAP, TBP, and Surakav, especially for DNN attacks DF and Var-CNN. In the closed-world setting, PEZD is at least 20 % less accurate than FRONT, WTF-PAD, FRONT-U, AAP, TBP, and Surakav for the DF attack, and for the Var-CNN attack, this advantage is more significant. In the open-world setting, PEZD achieves lower precision and recall at any threshold than FRONT, WTF-PAD, and Surakav.

One of the questions our study raises is why PEZD performs so well. First, PEZD significantly increases the difference among intra-website traffic traces by adding dummy packets with distinct distributions into each trace. This renders attackers facing challenges in learning common patterns within the website. Furthermore, PEZD introduces significant confusion between different website categories, posing substantial challenges to website recognition.

We apply Cosine similarity to analyze intra-similarity, where the Cosine distance is calculated for different traffic traces at the same website.

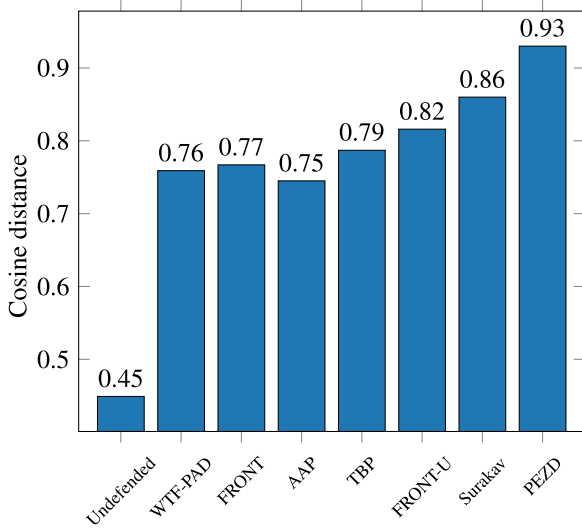


Fig. 14. Calculate intra-similarity for undefined, WTF-PAD, FRONT, FRONT-U, Surakav, AAP, TBP, and PEZD datasets based on the Cosine distance.

The Cosine similarity is shown in Eq. (3),

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n (A_i)^2} \times \sqrt{\sum_{i=1}^n (B_i)^2}} \quad (3)$$

where A and B are vectors. Cosine distance $D = 1 - S(A, B)$, where S is Cosine similarity. A larger Cosine distance represents a larger difference for a given pair of traffic traces. We randomly select one hundred pairs of traffic traces from a website to calculate the Cosine distance and average it as the Cosine distance of the website. We average the Cosine distance of all websites as a final difference representation for undefined, WTF-PAD, FRONT, Surakav, and PEZD datasets. The DS-19 dataset is used for the experiments. Cosine distances are shown in Fig. 14.

The cosine distance between traffic traces of the same website for PEZD is larger than for FRONT, WTF-PAD, AAP, TBP, FRONT-U, and Surakav. Specifically, the cosine distance of PEZD is 0.16, 0.17, 0.18, 0.14, 0.11, and 0.07 higher than that of FRONT, WTF-PAD, AAP, TBP, FRONT-U, and Surakav, respectively. Significant differences between traffic traces of the website can hinder attackers from discovering unique patterns in the website.

In addition, we use the non-linear dimensionality reduction algorithm t-SNE [50] to visualize the effect of defenses on the inter-websites. T-SNE embeds high-dimensional features into a low-dimensional space. We map each traffic trace to a 2D space, as shown in Fig. 15. We can observe that website categories are clearly separated for the undefined traffic, and traffic traces from the same website exhibit cohesion. In contrast, PEZD greatly disrupts the clustering of the same website, with a haphazard distribution of traffic traces. On the other hand, WTF-PAD, FRONT, Surakav, AAP, TBP, and FRONT-U fail to disperse the traffic traces completely, and some traffic traces from the same website continue to exhibit aggregation. Specifically, for WTF-PAD, Surakav, FRONT, and TBP, the traffic traces of label 32 demonstrate cohesion. Furthermore, for Surakav, traffic traces labeled 50, and 78 exhibit strong cohesion. While for FRONT, it is more obvious that traffic traces from the same category are aggregated.

To understand how PEZD affects the feature importance in the classifier's decision-making process, we conducted a LIME analysis (see Appendix A).

6.2. PEZD traffic trace pool maintenance

PEZD requires collecting traffic traces to generate defense information, whether the traffic traces in the traffic trace pool need to be up-

Table 5

Impact of historical datasets as the traffic trace pool on the accuracy of WF attacks.

Traffic trace pool	Bandwidth	Var-CNN	DF	CUMUL
DS-19 dataset	43.39 %	19.39 %	37.88 %	21.45 %
DS-14 dataset	44.54 %	24.86 %	46.83 %	25.69 %

dated regularly to ensure the effectiveness of the defense during deployment. Intuitively, PEZD leverages the differences between different websites, and these differences do not change after they are collected. Therefore, it still provides good protection performance over time. To validate our idea, we use the DS-14 dataset as the traffic trace pool and the DS-19 dataset as user traffic. These two datasets were collected approximately five years apart.

Table 5 shows the evaluation results. PEZD's protection success rate dropped by less than 5% for the CUMUL attack. Even against state-of-the-art DNN attacks, the PEZD defense success rate dropped by less than 10%. This suggests that PEZD does not require significant effort to update traffic traces during deployment regularly. Our explanation for the degraded performance of PEZD is that the difference in traffic traces in the DS-14 dataset is smaller than in the DS-19 dataset. We calculate Cosine distances of 0.76 and 0.69 for the DS-19 and DS-14 datasets, respectively. The smaller differences between traffic traces may lead to smaller differences in the distribution of dummy packets. Note that we calculate the difference between unmonitored websites since we select unmonitored websites as the traffic trace pool.

6.3. Implementation

In this section, we will discuss potential problems in the implementation and give explanations.

Traffic trace pool: PEZD implementation defense requires the assistance of a traffic trace pool, which, with its size of a few tens of MB, can be placed in the Tor directory server. When a user requests relay information from the Tor directory server, the location information of the dummy packet is sent to the user along with the relay information. Alternatively, the traffic trace pool can be placed within clients and Tor bridges to generate dummy packet location information. In addition, we can utilize existing tools such as Tor Browser Selenium [51], which is a Python library that allows for automated operation of the Tor Browser using selenium drivers. Furthermore, collecting these traffic traces does not impose a significant network burden on the Tor network. Tor is used by millions of people every day, and some websites need to handle millions of requests, while we only need to collect 10,000 traffic traces. Furthermore, our study also shows that these collected traces do not need to be updated frequently.

Implementation cost: PEZD only needs to use less than 2 KB of data to describe the extracted location information, which is very feasible for Tor. Tor client needs to load about 8 MB of data when starting up Tor for relay discovery [28]. In addition, we calculate that the computational cost required to extract all dummy packet location information on a CPU (Intel(R) Core(TM) i7-12700H laptop) is about 0.06 milliseconds. Note that the Tor directory server can generate location information before the user requests it, which does not add latency. In contrast, Surakav requires some 186 milliseconds to generate reference traffic traces on the same CPU. In terms of computational overhead, PEZD is 3000× lower than Surakav. In addition, the generation and removal of dummy packets operate at the network layer. In practical implementation, this process can be facilitated by leveraging obfs4proxy [52], a pluggable transport [53] protocol provided by Tor. It is important to note that these dummy packets do not impact real network communication.

6.4. Limitations and future work

With the application of DNN models to WF attacks, most DNN-based attacks use only the packet direction sequences as the input to models.

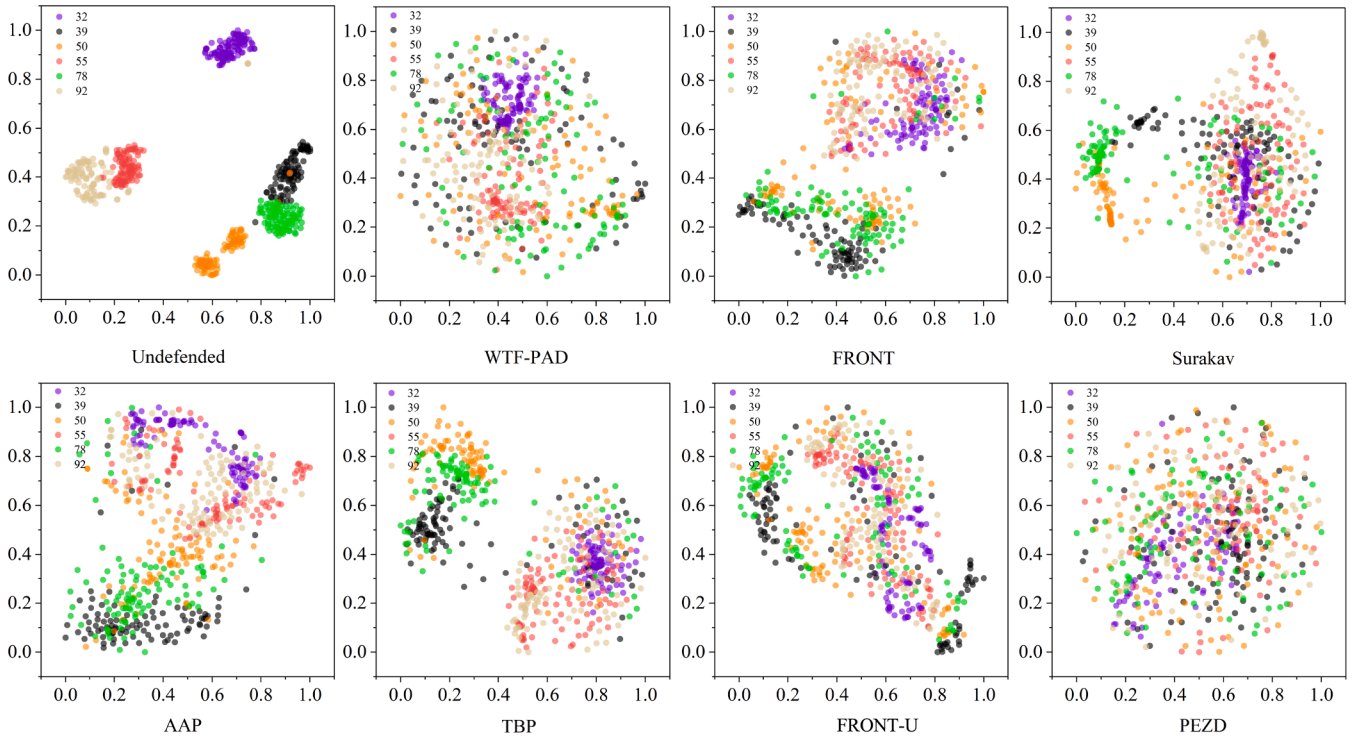


Fig. 15. T-SNE visualization for undefended, WTF-PAD, FRONT, Surakav, AAP, TBP, FRONT-U, and PEZD. Each color represents one website class. The number indicates the label in the dataset.

This also leads most defenses to focus only on the direction of packets, ignoring packet timing information [18,22–25,28,29,45]. Some recent works [10,13] utilize the timing features of packets to identify the websites visited by users. Intuitively, PEZD injection of dummy packets does destroy the timing features of traffic traces. However, PEZD cannot generate accurate dummy packet timestamps. As a result, it cannot be evaluated for its effectiveness in defending against WF attacks that use timing features. Although our current defense cannot be directly applied to WF attacks utilizing timing features, extensions to address such attacks can be developed by optimizing the transmission timing of dummy packets or by incorporating the Adaptive Padding [39] algorithm to disrupt timing features. Future work will investigate the joint perturbation of packet direction and timing features.

Additionally, some traditional attacks also use burst features as the feature set [6]. While not explicitly designed to do so, PEZD does impact burst features to some extent. The addition of dummy packets increases the size of existing bursts in the original traffic and creates new burst features. For stronger perturbation burst features, we can extend our approach to disrupt the feature by modifying a previous single dummy packet location to send dummy packets at successive locations in the same direction.

Finally, although previous probability-based methods for generating dummy packets with different distributions have not been significantly successful, this does not preclude the possibility that probability can be used in conjunction with or in place of our method to further enhance defense performance. We remain open to exploring probabilistic approaches in our future work.

7. Conclusion

In this paper, we propose PEZD, a practical and effective zero-delay defense. PEZD aims to hinder attackers from discerning a website's unique patterns by injecting dummy packets with diverse distributions into the traffic. It exploits the distribution of outgoing packets from different websites to randomize the distribution of added dummy packets.

PEZD offers better protection performance than FRONT, WTF-PAD, and Surakav with similar overhead. In the closed-world setting, it reduces the accuracy of the state-of-the-art attack, Var-CNN, from 97.56 % to 19.39 %, while FRONT only reduces it to 82.92 %, WTF-PAD to 93.93 % and Surakav to 69.93 %. In the open-world setting, PEZD also shows lower precision and recall than FRONT, WTF-PAD, and Surakav. Furthermore, PEZD supports the injection of dummy packets only on the client-side for effective defense. In addition, we also show that PEZD does not require significant effort to maintain during deployment.

CRedit authorship contribution statement

Hengheng Xiong: Data curation, Conceptualization, Formal analysis, Methodology; **Dapeng Man:** Supervision; **Huanran Wang:** Writing – review & editing; **Jingwen Tan:** Writing – review & editing; **Jiguang Lv:** Writing – review & editing; **Wu Yang:** Supervision.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This project was supported by National Key R&D Program of China (Grant No. 2021YFB3101401), Natural Science Foundation of Heilongjiang Province of China (Grant No. TD2022F001), Joint Funds of the [National Natural Science Foundation of China](#) (Grant No. U2003206, No. U20B2048, No. U21B2019, No. U22A2036), [National Natural Science Foundation of China](#) (Grant No. 62272127).

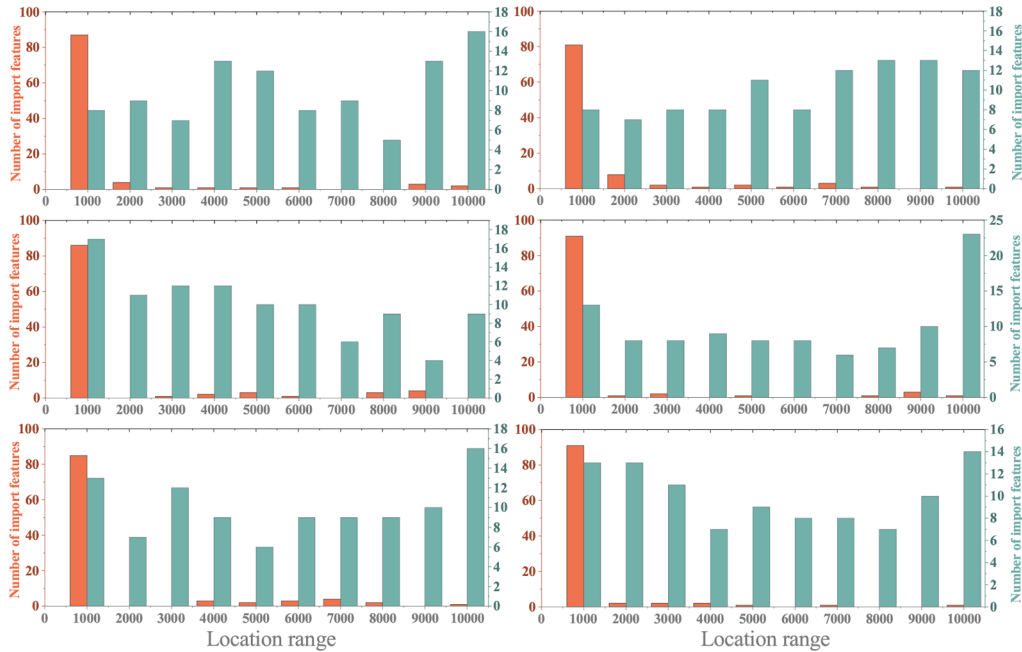


Fig. A.1. Visualization of the number of important features within intervals. The number of feature importance indicates how many significant features fall within an interval, with each interval having a length of 1000.

Appendix A. Analyzing PEZD Using LIME

We randomly selected six samples from category 8 to showcase the LIME results. For each traffic trace, we considered individual packets (+1 or −1) as features and analyzed the range of the top 100 most important features, as shown in Fig. A.1.

The LIME analysis reveals a stark contrast between undefended and PEZD-defended traffic. For undefended traffic, 80 % of the most important features are concentrated within the first 1000 packets. This observation clearly indicates that website fingerprinting attacks predominantly rely on the header features of the traffic. In contrast, traffic defended by PEZD shows no discernible trend of dependence on specific features. The distribution of the most important features varies significantly across different traffic traces. This reflects the effectiveness of PEZD from the side, that is, it is difficult for the model to learn the specific features of traffic to generalize to other traffic sequences in the same category.

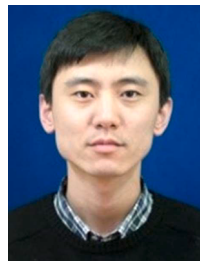
References

- [1] R. Dingledine, N. Mathewson, P.F. Syverson, Tor: the second-generation onion router, in: Proceedings of the 13th USENIX Security Symposium, August 9-13, 2004, San Diego, CA, USA, USENIX, 2004, pp. 303–320.
- [2] A. Mani, T. Wilson-Brown, R. Jansen, A. Johnson, M. Sherr, Understanding tor usage with privacy-preserving measurement, in: Proceedings of the Internet Measurement Conference 2018, IMC 2018, Boston, MA, USA, October 31 - November 02, 2018, ACM, 2018, pp. 175–187.
- [3] Tor Project, Users - tor metrics, <https://metrics.torproject.org/userstats-relay-country.html>, Last accessed on 2023-05-06.
- [4] X. Cai, X.C. Zhang, B. Joshi, R. Johnson, Touching from a distance: website fingerprinting attacks and defenses, in: Proceedings of the 2012 ACM Conference on Computer and Communications Security, CCS '12, Association for Computing Machinery, New York, NY, USA, 2012, p. 605–616. <https://doi.org/10.1145/2382196.2382260>
- [5] T. Wang, I. Goldberg, Improved website fingerprinting on Tor, in: Proceedings of the 12th annual ACM Workshop on Privacy in the Electronic Society, WPES 2013, Berlin, Germany, November 4, 2013, ACM, 2013, pp. 201–212. <https://doi.org/10.1145/2517840.2517851>
- [6] T. Wang, X. Cai, R. Nithyanand, R. Johnson, I. Goldberg, Effective attacks and provable defenses for website fingerprinting, in: Proceedings of the 23rd USENIX Security Symposium, San Diego, CA, USA, August 20-22, 2014, USENIX Association, 2014, pp. 143–157.
- [7] J. Hayes, G. Danezis, k-Fingerprinting: a robust scalable website fingerprinting technique, in: 25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016, USENIX Association, 2016, pp. 1187–1203.
- [8] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, K. Wehrle, Website fingerprinting at internet scale, in: 23rd Annual Network and Distributed System Security Symposium, NDSS 2016, San Diego, California, USA, February 21-24, 2016, The Internet Society, 2016.
- [9] P. Sirinam, M. Imani, M. Juarez, M. Wright, Deep fingerprinting: undermining website fingerprinting defenses with deep learning, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018, ACM, 2018, pp. 1928–1943. <https://doi.org/10.1145/3243734.3243768>
- [10] S. Bhat, D. Lu, A. Kwon, S. Devadas, Var-CNN: a data-efficient website fingerprinting attack based on deep learning, Proc. Priv. Enhancing Technol. 2019 (4) (2019) 292–310. <https://doi.org/10.2478/POPET-2019-0070>
- [11] V. Rimmer, D. Preuveneers, M. Juarez, T. van Goethem, W. Joosen, Automated website fingerprinting through deep learning, in: 25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018, The Internet Society, 2018.
- [12] D. Herrmann, R. Wendolsky, H. Federrath, Website fingerprinting: attacking popular privacy enhancing technologies with the multinomial naïve-bayes classifier, in: Proceedings of the First ACM Cloud Computing Security Workshop, CCSW 2009, Chicago, IL, USA, November 13, 2009, ACM, 2009, pp. 31–42. <https://doi.org/10.1145/1655008.1655013>
- [13] M.S. Rahman, P. Sirinam, N. Mathews, K.G. Gangadhara, M. Wright, Tik-Tok: the utility of packet timing in website fingerprinting attacks, Proc. Privacy Enhanc. Technol. 2020 (3) (2020) 5–24. <https://doi.org/10.2478/POPET-2020-0043>
- [14] R. Nithyanand, X. Cai, R. Johnson, Glove: a bespoke website fingerprinting defense, in: Proceedings of the 13th Workshop on Privacy in the Electronic Society, WPES 2014, Scottsdale, AZ, USA, November 3, 2014, ACM, 2014, pp. 131–134. <https://doi.org/10.1145/2665943.2665950>
- [15] C.V. Wright, S.E. Coull, F. Monrose, Traffic morphing: an efficient defense against statistical traffic analysis, in: Proceedings of the Network and Distributed System Security Symposium, NDSS 2009, San Diego, California, USA, 8th February - 11th February 2009, The Internet Society, 2009.
- [16] X. Luo, P. Zhou, E.W.W. Chan, W. Lee, R.K.C. Chang, R. Perdisci, HTTPoS: sealing information leaks with browser-side obfuscation of encrypted flows, in: Proceedings of the Network and Distributed System Security Symposium, NDSS 2011, San Diego, California, USA, 6th February - 9th February 2011, The Internet Society, 2011.
- [17] M. Juarez, M. Imani, M. Perry, C. Díaz, M. Wright, Toward an efficient website fingerprinting defense, in: Computer Security - ESORICS 2016 - 21st European Symposium on Research in Computer Security, Heraklion, Greece, September 26-30, 2016, Proceedings, Part I, 9878 of Lecture Notes in Computer Science, Springer, 2016, pp. 27–46. https://doi.org/10.1007/978-3-319-45744-4_2
- [18] M.S. Rahman, M. Imani, N. Mathews, M. Wright, Mockingbird: defending against deep-learning-based website fingerprinting attacks with adversarial traces, IEEE Trans. Inf. Forensics Secur. 16 (2021) 1594–1609. <https://doi.org/10.1109/TIFS.2020.3039691>
- [19] K.P. Dyer, S.E. Coull, T. Ristenpart, T. Shrimpton, Peek-a-boo, I still see you: why efficient traffic analysis countermeasures fail, in: IEEE Symposium on Security and Privacy, SP 2012, 21-23 May 2012, San Francisco, California, USA, IEEE Computer Society, 2012, pp. 332–346. <https://doi.org/10.1109/SP.2012.28>

- [20] X. Cai, R. Nithyanand, T. Wang, R. Johnson, I. Goldberg, A systematic approach to developing and evaluating website fingerprinting defenses, in: G. Ahn, M. Yung, N. Li (Eds.), Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, Scottsdale, AZ, USA, November 3-7, 2014, ACM, 2014, pp. 227–238. <https://doi.org/10.1145/2660267.2660362>
- [21] X. Cai, R. Nithyanand, R. Johnson, CS-BuFLO: a congestion sensitive website fingerprinting defense, in: Proceedings of the 13th Workshop on Privacy in the Electronic Society, WPES 2014, Scottsdale, AZ, USA, November 3, 2014, ACM, 2014, pp. 121–130. <https://doi.org/10.1145/2665943.2665949>
- [22] A.M. Sadeghzadeh, B. Tajali, R. Jalili, AWA: adversarial website adaptation, IEEE Trans. Inf. Forensics Secur. 16 (2021) 3109–3122. <https://doi.org/10.1109/TIFS.2021.3074295>
- [23] S. Shan, A.N. Bhagoji, H. Zheng, B.Y. Zhao, Patch-based defenses against web fingerprinting attacks, in: AISec@CCS 2021: Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security, Virtual Event, Republic of Korea, 15 November 2021, ACM, 2021, pp. 97–109. <https://doi.org/10.1145/3474369.3486875>
- [24] D. Li, Y. Zhu, M. Chen, J. Wang, Minipatch: undermining DNN-based website fingerprinting with adversarial patches, IEEE Trans. Inf. Forensics Secur. 17 (2022) 2437–2451. <https://doi.org/10.1109/TIFS.2022.3186743>
- [25] C. Hou, G. Gou, J. Shi, P. Fu, G. Xiong, WF-GAN: fighting back against website fingerprinting attack using adversarial learning, in: IEEE Symposium on Computers and Communications, ISCC 2020, Rennes, France, July 7-10, 2020, IEEE, 2020, pp. 1–7. <https://doi.org/10.1109/ISCC50000.2020.9219593>
- [26] A. Panchenko, L. Niessen, A. Zinnen, T. Engel, Website fingerprinting in onion routing based anonymization networks, in: Proceedings of the 10th Annual ACM Workshop on Privacy in the Electronic Society, WPES 2011, Chicago, IL, USA, October 17, 2011, ACM, 2011, pp. 103–114. <https://doi.org/10.1145/2046556.2046570>
- [27] J. Gong, T. Wang, Zero-delay lightweight defenses against website fingerprinting, in: 29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020, USENIX Association, 2020, pp. 717–734.
- [28] T. Wang, I. Goldberg, Walkie-Talkie: an efficient defense against passive website fingerprinting attacks, in: 26th USENIX Security Symposium, USENIX Security 2017, Vancouver, BC, Canada, August 16-18, 2017, USENIX Association, 2017, pp. 1375–1390.
- [29] M. Nasr, A. Bahramali, A. Houmansadr, Defeating DNN-based traffic analysis systems in real-time with blind adversarial perturbations, in: 30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021, USENIX Association, 2021, pp. 2705–2722.
- [30] J. Gong, W. Zhang, C. Zhang, T. Wang, Surakav: generating realistic traces for a strong website fingerprinting defense, in: 43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022, IEEE, 2022, pp. 1558–1573. <https://doi.org/10.1109/SP46214.2022.9833722>
- [31] J.K. Holland, N. Hopper, RegulaTor: a straightforward website fingerprinting defense, Proceedings on Privacy Enhancing Technologies 2022 (2) (2022) 344–362. <https://doi.org/10.2478/POPETs-2022-0049>
- [32] Y. Wang, Y. Wang, J.J. Huang, Y. Chen, TBP: tree structure burst-sequence padding defense against website fingerprinting, in: 2023 IEEE International Performance, Computing, and Communications Conference (IPCCC), IEEE, 2023, pp. 99–108.
- [33] N. Mathews, J.K. Holland, S.E. Oh, M.S. Rahman, N. Hopper, M. Wright, SoK: a critical evaluation of efficient website fingerprinting defenses, in: 44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023, IEEE, 2023, pp. 969–986. <https://doi.org/10.1109/SP46215.2023.10179289>
- [34] L. Qiao, B. Wu, S. Yin, H. Li, W. Yuan, X. Luo, Resisting DNN-based website fingerprinting attacks enhanced by adversarial training, IEEE Trans. Inform. Forensics Secur. 18 (2023) 5375–5386. <https://doi.org/10.1109/TIFS.2023.3304528>
- [35] T. Dahanayaka, G. Jourjon, S. Seneviratne, Dissecting traffic fingerprinting CNNs with filter activations, Comput. Netw. 206 (2022) 108770. <https://doi.org/10.1016/J.COMNET.2022.108770>
- [36] Z. Yang, X. Xiao, B. Zhang, G. Hu, Q. Li, Q. Liu, AAP: defending against website fingerprinting through burst obfuscation, in: International Conference on Advanced Data Mining and Applications, Springer, 2023, pp. 107–121.
- [37] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016, IEEE Computer Society, 2016, pp. 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [38] X. Gu, B. Song, W. Lan, M. Yang, An online website fingerprinting defense based on the non-targeted adversarial patch, Tsinghua Sci. Technol. 28 (6) (2023) 1148–1159. <https://doi.org/10.26599/TST.2023.9010062>
- [39] V. Shmatikov, M. Wang, Timing analysis in low-latency mix networks: attacks and defenses, in: Computer Security - ESORICS 2006, 11th European Symposium on Research in Computer Security, Hamburg, Germany, September 18-20, 2006, Proceedings, 4189 of Lecture Notes in Computer Science, Springer, 2006, pp. 18–33. https://doi.org/10.1007/11863908_2
- [40] M. Perry, G. Kadianakis, Circuit padding developer documentation, <https://github.com/torproject/tor/blob/main/doc/HACKING/CircuitPaddingDevelopment.md>, Last accessed on 2023-05-06.
- [41] W. De la Cadena, A. Mitseva, J. Hiller, J. Pennekamp, S. Reuter, J. Filter, T. Engel, K. Wehrle, A. Panchenko, TrafficSliver: fighting website fingerprinting attacks with traffic splitting, in: CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020, ACM, 2020, pp. 1971–1985. <https://doi.org/10.1145/3372297.3423351>
- [42] S. Henri, G. Garcia-Aviles, P. Serrano, A. Banchs, P. Thiran, Protecting against website fingerprinting with multihoming, Proc. Privacy Enhanc. Technol. 2020 (2) (2020) 89–110. <https://doi.org/10.2478/POPETs-2020-0019>
- [43] M. Wang, A. Kulshrestha, L. Wang, P. Mittal, Leveraging strategic connection migration-powered traffic splitting for privacy, Proc. Privacy Enhanc. Technol. 2022 (3) (2022) 498–515. <https://doi.org/10.56553/POPETs-2022-0083>
- [44] M. Juarez, S. Afroz, G. Acar, C. Díaz, R. Greenstadt, A critical evaluation of website fingerprinting attacks, in: Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, Scottsdale, AZ, USA, November 3-7, 2014, ACM, 2014, pp. 263–274. <https://doi.org/10.1145/2660267.2660368>
- [45] A. Abusnaina, R. Jang, A. Khormali, D. Nyang, D. Mohaisen, DFD: adversarial learning-based approach to defend against website fingerprinting, in: 39th IEEE Conference on Computer Communications, INFOCOM 2020, Toronto, ON, Canada, July 6-9, 2020, IEEE, 2020, pp. 2459–2468. <https://doi.org/10.1109/INFOCOM41043.2020.9155465>
- [46] G. Srivastava, R.H. Jha, S. Bhattacharya, S. Pandya, Rajeswari, P.K.R. Madrikunta, G. Yenduri, J.G. Hall, M. Alazab, T.R. Gadekallu, XAI for cybersecurity: state of the art, challenges, open issues and future directions, CoRR (2022). <https://doi.org/10.48550/ARXIV.2206.03585>
- [47] M.T. Ribeiro, S. Singh, C. Guestrin, “Why should I trust you?": Explaining the predictions of any classifier, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016, ACM, 2016, pp. 1135–1144. <https://doi.org/10.1145/2939672.2939778>
- [48] Alexa, The Web Information Company, <https://www.alexa.com>, Last accessed on 2024-10-21.
- [49] J. Smith, L. Dolfi, P. Mittal, A. Perrig, QCSd: a QUIC client-side website fingerprinting defence framework, in: 31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10-12, 2022, USENIX Association, 2022, pp. 771–789.
- [50] L. van der Maaten, G. Hinton, Visualizing data using t-SNE, J. Mach. Learn. Res. 9 (86) (2008) 2579–2605.
- [51] G. Acar, M. Juarez, i. contributors, tor-browser-selenium - Tor Browser automation with Selenium, 2023, (<https://github.com/webfp/tor-browser-selenium>).
- [52] Yawning Angel, obfs4 - The obfourscator, <https://github.com/Yawning/obfs4>, Last accessed on 2024-10-21.
- [53] Tor Project, pluggable transports, <https://support.torproject.org/glossary/pluggable-transports/>, Last accessed on 2024-10-21.



Hengheng Xiong is currently pursuing the Ph.D. degree with the College of Computer Science and Technology, Harbin Engineering University. His main research interests include network security and artificial intelligence security.



Dapeng Man received the B.S. and M.S. and Ph.D. degrees in computer science and technology from Harbin Engineering University in 2003, 2007 and 2009. He is currently an professor in Harbin Engineering University. His main research interests include mobile computing security and IoT security. He is a Member of CCF.



Huanran Wang received the bachelor's degree from the School of Information Science and Technology, Southwest Jiaotong University, Chengdu, China, in 2011, and the master's degree from the School of Computer Science and Technology, Harbin Institute of Technology, Harbin, China, in 2016. He is currently an associate professor in Harbin Engineering University, Harbin Engineering University, Harbin, China. His main research interests include social networks, privacy preserving and representation learning.



Jingwen Tan is currently pursuing the Ph.D. degree with the College of Computer Science and Technology, Harbin Engineering University. Her main research interests include encrypted traffic classification and website fingerprinting.



Wu Yang received a Ph.D. degree in Computer System Architecture Specialty of Computer Science and Technology School from Harbin Institute of Technology. He is currently a professor and doctoral supervisor of Harbin Engineering University. His main research interests include wireless sensor network, peer-to-peer network and information security. He is a Member of ACM and Senior Member of CCF.



Jiguang Lv received the M.S. and Ph.D. degrees in computer science and technology from Harbin Engineering University in 2014 and 2018. He is currently an associate professor in Harbin Engineering University. His main research interests include mobile computing security and IoT security. He is a Member of CCF.